

z/OS Addressing Modes, Address Spaces, LPARs, SYSPLEX





SOME KEY CONCEPTS



Even at this early stage, you're probably starting to feel as if z/OS is a massive bowl of alphabet soup comprising hundreds of components with cryptic names and mysterious functions.

Yes, that's about right.

Toto, I don't think we're in Linux anymore.

Let's take a look at a few key concepts underlying the architecture and operation of Z systems. Some of the alphabet soup will start to make sense, one spoonful at a time.

- Addressing Modes
- Address Spaces
- Logical Partitions
- Parallel Sysplex

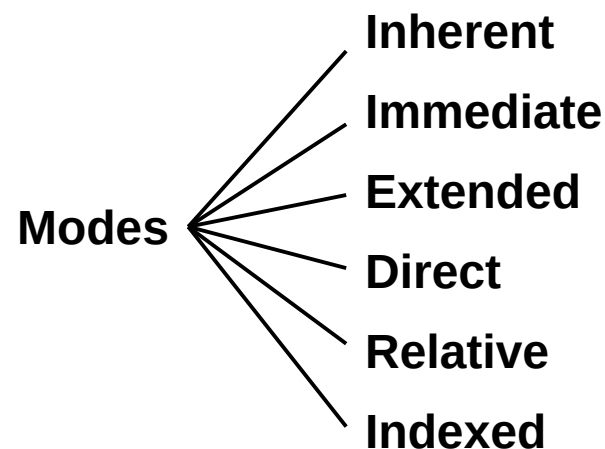
ADDRESSING MODES

In our line of work we're faced with a bewildering array of buzzwords, abbreviations, and acronyms. As if that weren't enough, many of these terms are *overloaded* – that is, they have different meanings depending on context.

The term *addressing mode* has a handful of different meanings. You may already know one or more of these meanings through your previous experience with other computer systems. So, let's clarify what the term means in the context of z/OS.

In general, addressing modes are the ways in which an instruction calculates the location of its operands.

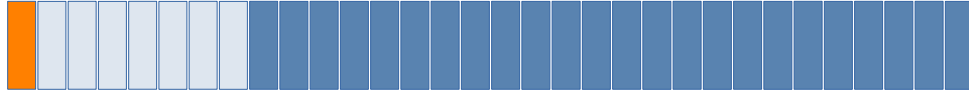
Ben Clifford, in a 2024 lecture about microcontroller architecture, describes these addressing modes:



That sort of taxonomy applies to all kinds of computer architectures, including the logical architecture of z/OS. But as is so often the case with z/OS, there's one more twist.

Z/OS ADDRESSING MODES

1964. System/360. 32-bit word. Doubleword. 24 bit address. 16MB max.



Instructions and data

1983. IBM 3081. 32-bit word. Doubleword. 31 bit address. 2GB max.



Instructions and data

2000. zSeries Model 900. 64-bit word. Quadword. 64 bit address. 16EB max.



Data only

WHY SUPPORT 24-BIT ADDRESSING?

Why not "just" port or require recompilation of 24-bit code to more-advanced architectures?

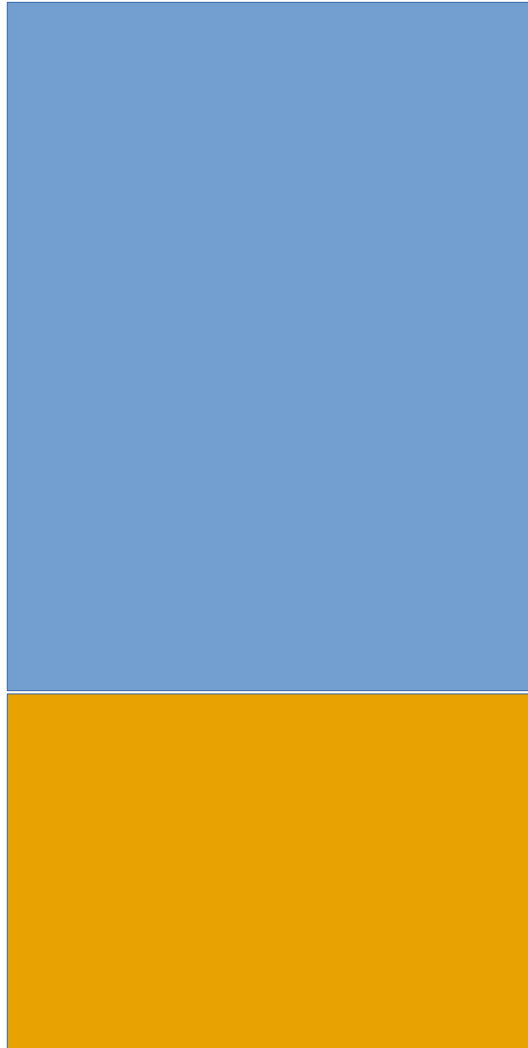
One reason is IBM's longstanding commitment to backward compatibility. Some customers have mission-critical applications in production for which the source code is long gone.

Another reason – possibly more interesting for Platform Engineers – is that the Brain-in-a-Jar came from a 24-bit body. Some of its deep internal functions are 24-bit. Its ganglia can only connect to 24-bit interfaces between the jar and the outside world.

So, the instruction set has 24-bit and 31-bit instructions that can access data using 24-bit and 31-bit addresses, and 31-bit instructions that can access data using 64-bit addresses.



ADDRESS SPACES



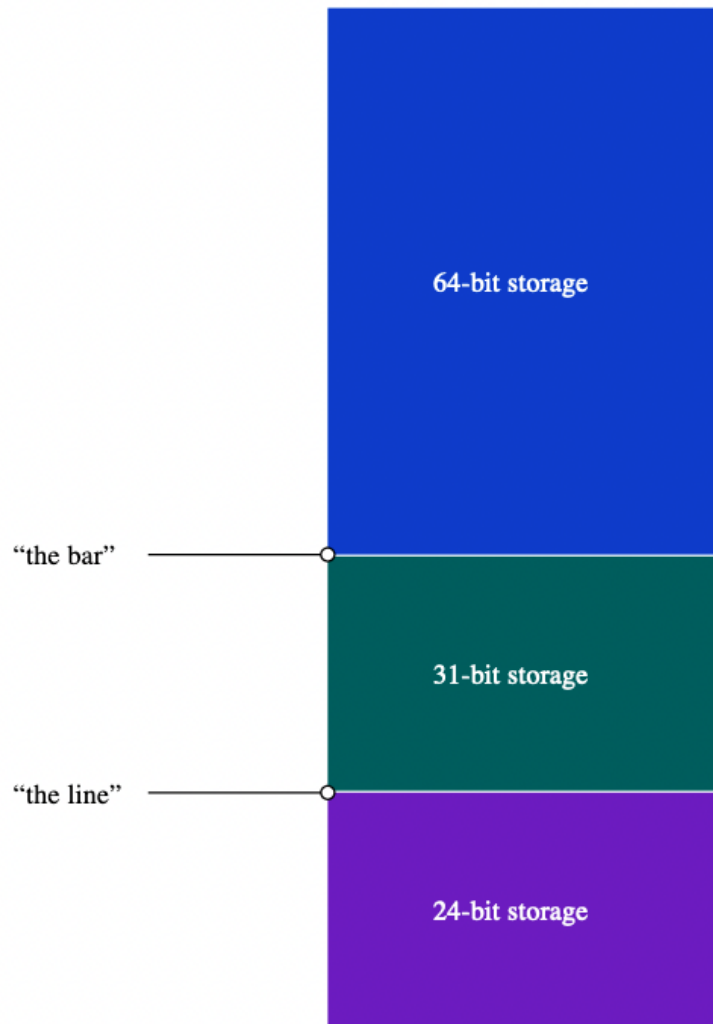
Most operating systems organize memory for applications in a way that isolates each program from others, while providing areas for the applications to interact with the operating system, and for the operating system to perform its functions.

On z/OS, address spaces are also called *regions*.

Private memory. Controlled by the process that's running in the address space.

Common memory. Virtual addresses in all address spaces point to the same areas. Used by the operating system.

Z/OS ADDRESS SPACE LAYOUT



To support backward compatibility for 24-bit addressing and to accommodate the implementation of 31-bit and 64-bit addressing modes, private and common areas in z/OS address spaces have to be divided into three parts.

Cool insider jargon: The Line, The Bar.

The line is the maximum value that can be represented in 24 bits.

The bar is the maximum value that can be represented in 31 bits.

Cool insider chatter:

"Those control blocks have to reside below the line."

"You can't put executable code above the bar."

"That program was complied with AMODE24. It can't see addresses above the line."

64-bit storage 31-bit storage 24-bit storage

Diagram from
<https://www.ibm.com/docs/en/cics-ts/6.x?topic=storage-understanding-zos-address-space>

Z/OS ADDRESS SPACE LAYOUT

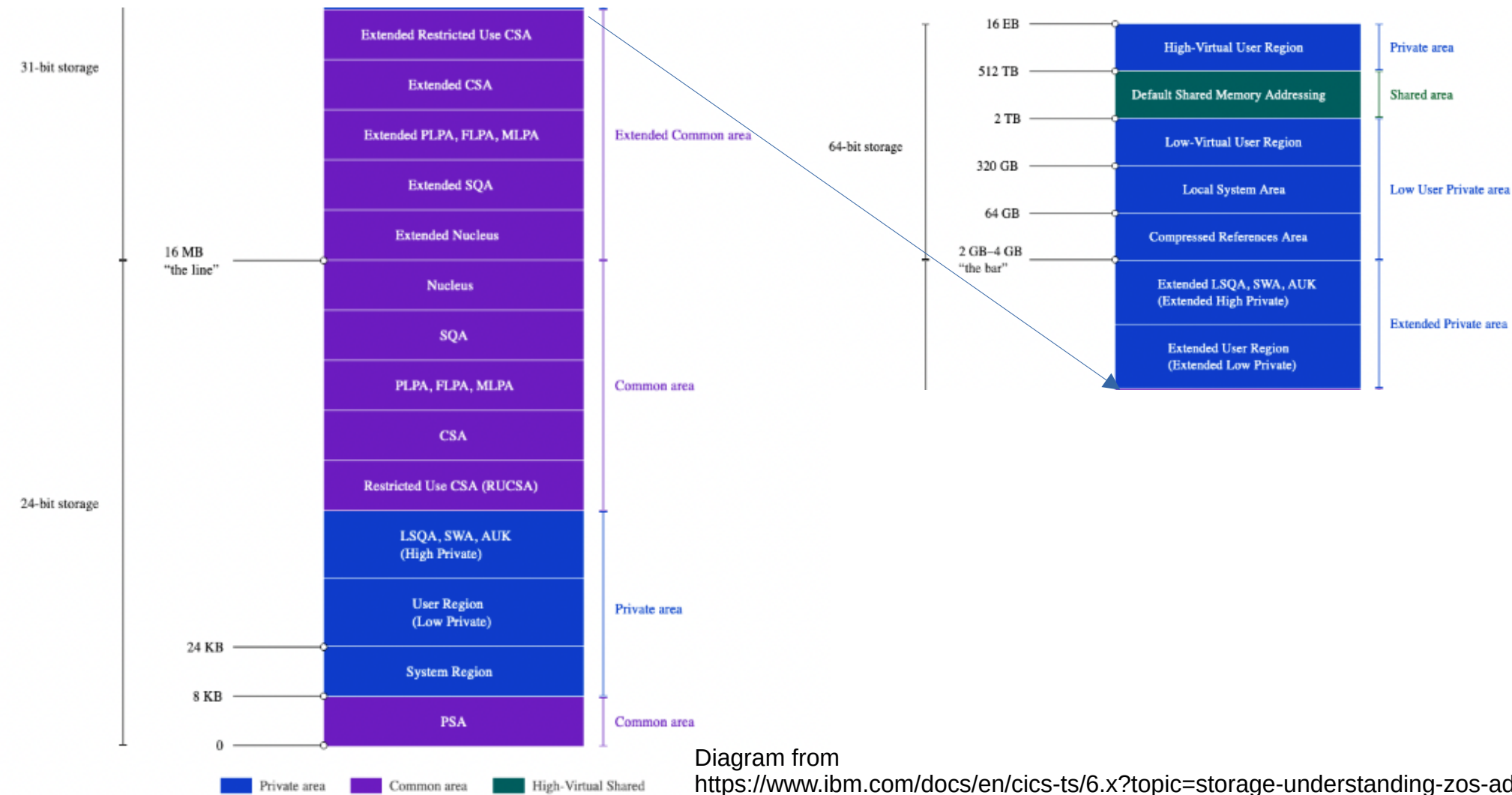


Diagram from <https://www.ibm.com/docs/en/cics-ts/6.x?topic=storage-understanding-zos-address-space>

Z/OS ADDRESS SPACE LAYOUT

Let's decode some of the alphabet soup on the previous slide.

But first we need to understand *storage protection keys*.

Each 4K segment of real memory has a 7-bit control field called a *storage key*. Meanwhile, each executing program has a 4-bit *protection key* in its Program Status Word (PSW).

Whenever a program accesses a real memory location (after translation from a virtual storage address), the system compares bits 0-3 of the *storage key* with the *protection key* in the PSW to determine whether the program is authorized to *fetch* (read) or *store* (write) that memory location.

The PSW protection key bits are assigned as follows:

- | | |
|-------|--|
| 0 | Supervisor and other system functions that require access to all storage |
| 1 | Job scheduler, JES2, APPC, TSO/E |
| 2 | Reserved |
| 3 | Availability Manager (AVM) |
| 4 | Reserved |
| 5 | Data Management and DAT |
| 6 | VTAM and DAT |
| 7 | IMS and DB2 |
| 8-9 | V=V problem programs (applications) |
| 10-15 | V=R problem programs (applications) |

Z/OS ADDRESS SPACE LAYOUT

Okay, now when you read or hear about "key 0" you'll know what it means. A program operating with key 0 has full access to the system's memory.

A "problem program" is an application program. It is meant to solve a business problem. (I know what you were thinking, there.)

A z/OS address space is divided into these areas:

All storage above the bar is called *high virtual storage*. It's accessible by programs running in 64-bit addressing mode (AMODE64). High virtual storage can contain data, but not executable code.

Extended areas reside above the line (64M) and below the bar (2GB). This is essentially a copy of the common areas below the line, for the convenience of programs operating in 31-bit or 64-bit mode.

The *Nucleus* is a key 0, read-only common storage area that contains low-level OS routines. Virtual addresses here map to the same real addresses. This is where the nucleus lives after IPL.

The SQA (System Queue Area) is a common area containing key 0 data accessed by multiple address spaces. The size is set at IPL time and can't be changed without a reboot, but if unused storage is available in the CSA the SQA can expand into that area dynamically.

The PLPA (Pageable Link Pack Area), FLPA (Fixed Link Pack Area), and MLPA (Modified Link Pack Area) contains system-level executable that are often called by applications in multiple address spaces. These are common areas located below the line.

Z/OS ADDRESS SPACE LAYOUT

The CSA (Common Storage Area) is available to all address spaces and is for application use. The size of the CSA is set at IPL time and can't be changed without an IPL, but some of it can be taken over by the SQA if the latter needs to expand.

LSQA, SWA, subpool 228, subpool 230 – various uses by system functions when they require address space level storage isolation. These areas are below the line.

User Region – accessible to application programs.

System Region – a small area for the use of the region control task in the address space.

PSA (Prefixed Save Area), also known as "low core." This constitutes addresses 0 through 8191 in each address space, and is associated with the particular hardware processor that is running the code in the address space at the moment.

Obligatory graphic to
break up all this text.



SYSTEM VIRTUAL STORAGE AREAS

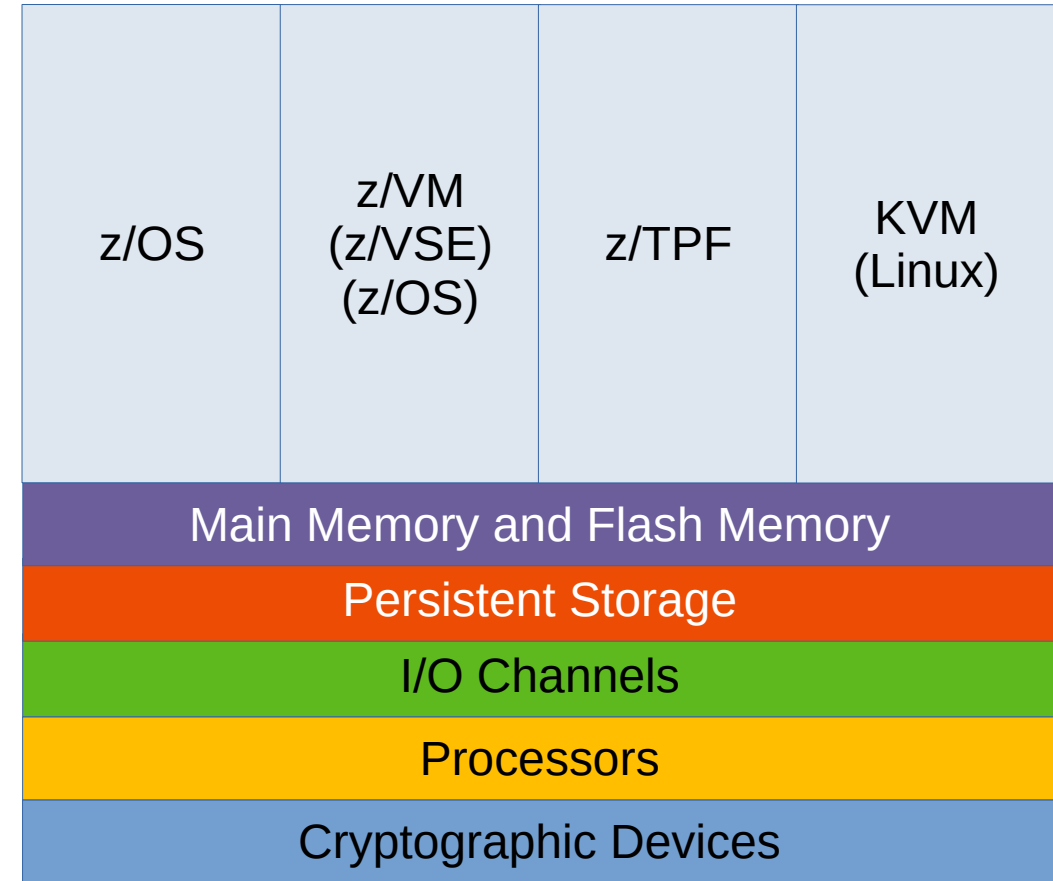
Area	Shared?	Pageable?	Purpose
CSA	Yes	Yes	Shared system structures
ECSA	Yes	Yes	Larger shared structures
SQA	Yes	No	Critical non-pageable system blocks
ESQA	Yes	No	Larger non-pageable blocks
Private	No	Yes	User programs & data
LSQA	No	Yes	Per-address-space control blocks
ELSQA	No	Yes	Larger per-address-space blocks
PLPA	Yes	Yes	Shared pageable system modules
FLPA	Yes	No	Shared fixed system modules
MLPA	Yes	Yes	IPL-only module overrides
Nucleus	Yes	No	Core OS Kernel

LOGICAL PARTITIONS (LPARS)

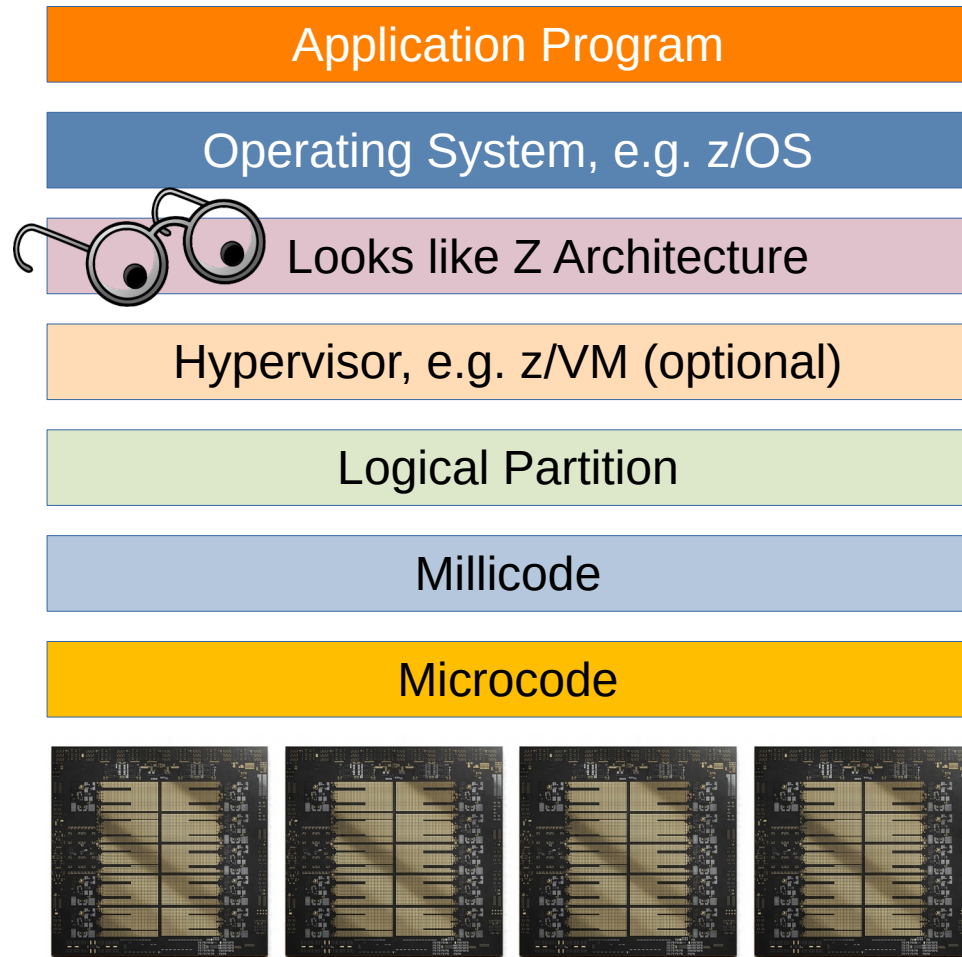
IBM Z systems can have over 200 processors (32-core, configurable), 42 TB of main memory, 6 TB of flash memory, multiple redundant I/O channels, and other physical components.

A Z can be subdivided into multiple logically-independent environments called *logical partitions*, or LPARs (pronounced "ell-par").

Each LPAR is assigned specific quantities of the various hardware elements available on the Z system. Each can be loaded with any of z/OS, z/VSE, z/TPF, Linux, or one of the Type 1 hypervisors z/VM or KVM. The Z supports SUSE Enterprise, Red Hat Enterprise, and Ubuntu Enterprise Linux.



LOGICAL PARTITIONS (LPARS)



Telum II processors (z17)

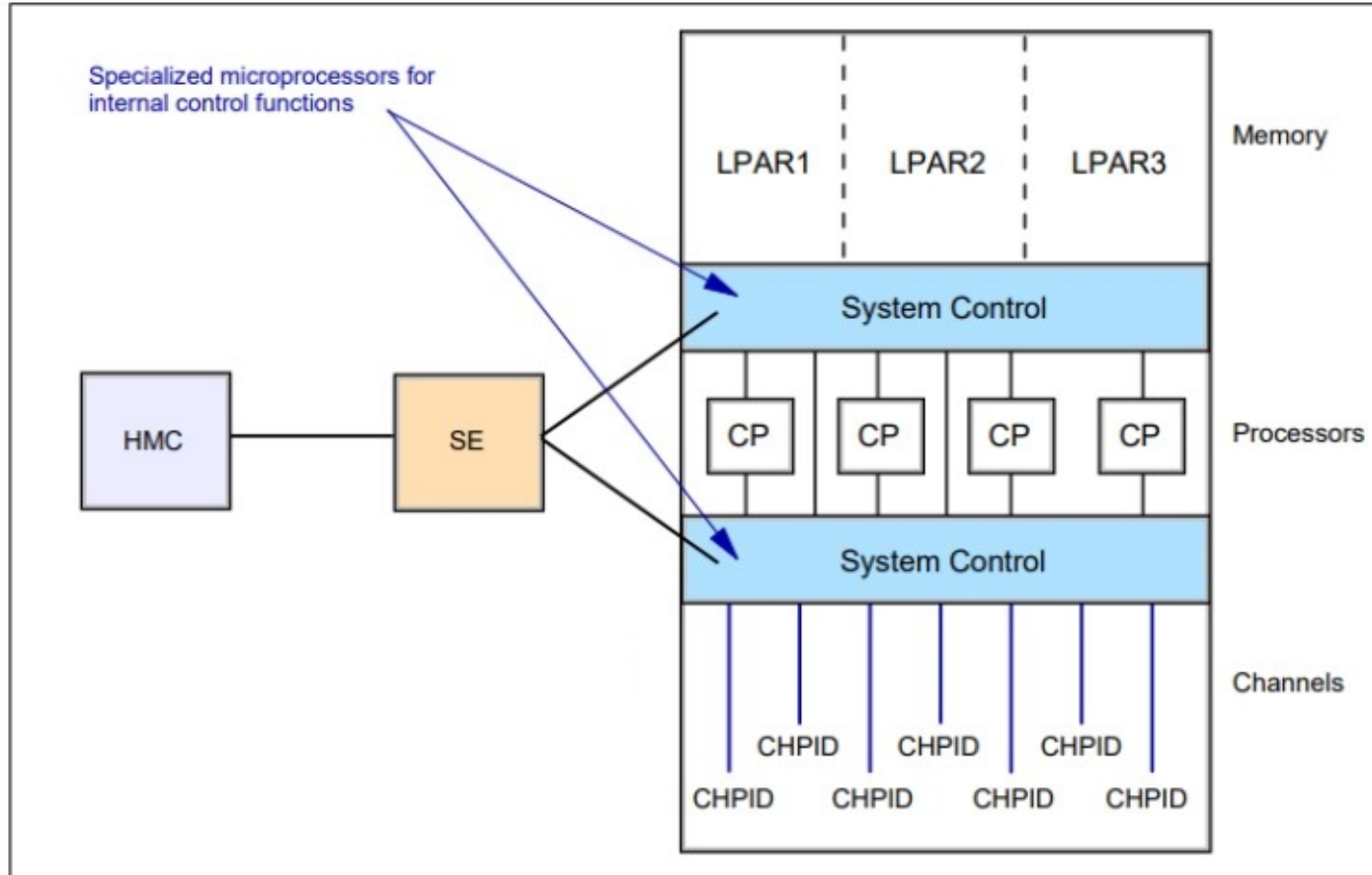
None of these operating systems can "see" the underlying hardware environment. Conceptually "above" the processors is a layer of horizontal microcode. Above that, there's a layer of vertical microcode. Those run in firmware.

Above that is a layer of millicode, a kind of vertical microcode that implements a superset of a subset of the instruction set documented in the *Z Architecture Principles of Operation*. That's as close to the "machine" as z/OS object code can get.

Above that is a layer of virtualization; the LPARs are themselves Type 1 hypervisors. When a program running under z/OS "thinks" it's running a documented instruction, it's actually several layers of abstraction removed from the processors.

In principle, "any" operating system can be ported to run on a Z, if it can be modified to work with the required interfaces.

LOGICAL PARTITIONS (LPARS)



If you have to have an LPAR to run an OS and you have to have an OS to run applications, how do you define and start up an LPAR?

The Hardware Management Console (HMC) is an HTTP application that connects to pre-installed System Control (CSP) software on the Z. It runs on specialized processors and not on the general processors used by applications.

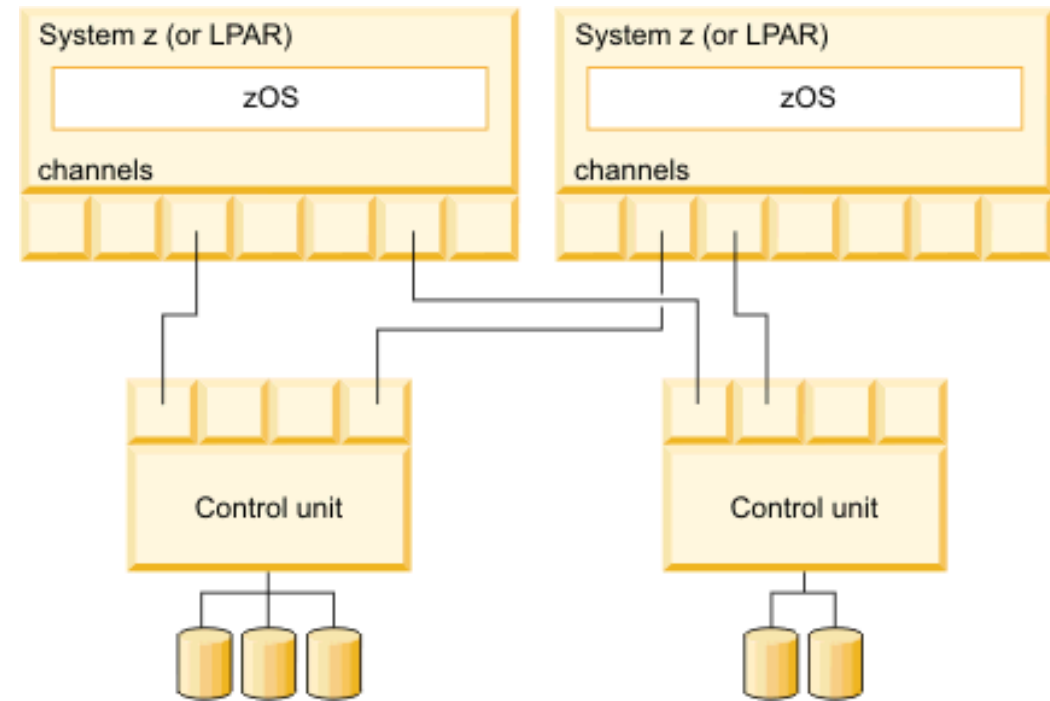
The System Element (SE) is a laptop installed inside the Z rack (two, for redundancy). One does not normally open the cabinet unless the system is in a badly failed state, but if you must, you must.

Image: <https://www.riskinsight-wavestone.com/en/2024/12/segmentation-in-mainframe-z-os-and-lpm/>

Z/OS CLUSTERING: SHARED DASD

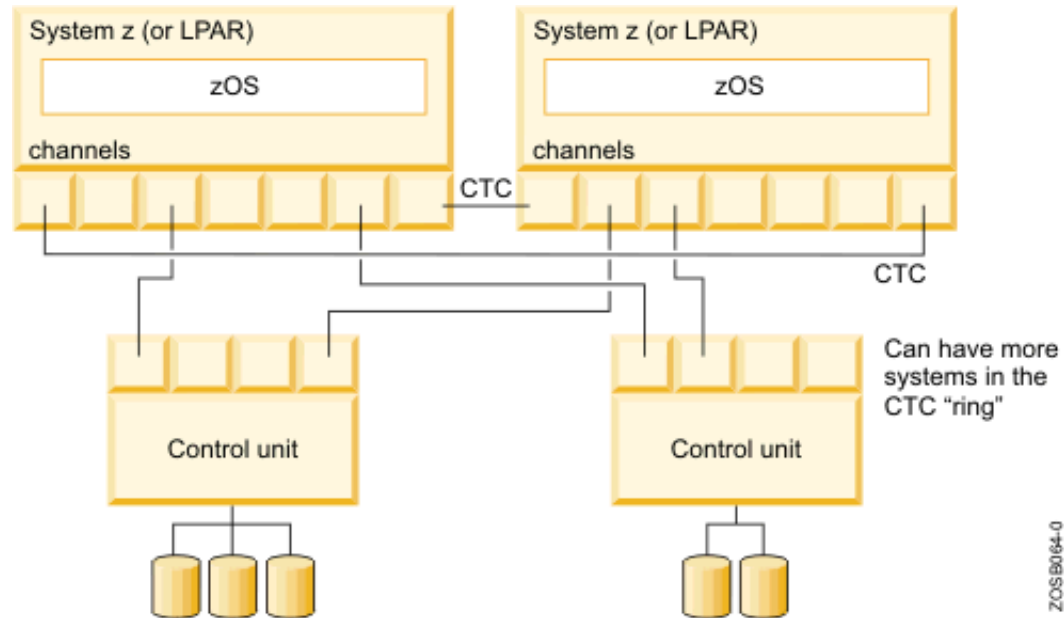
There are three levels of clustering technology for the IBM Z, each providing more robust support than the previous one.

At the most basic level, z/OS LPARs or physical Z systems can share DASD (Direct Access Storage Devices), or persistent storage devices that the Brain-in-a-Jar sees as DASD.



ZOSB063-0

Z/OS CLUSTERING: CTC RING



With Channel-to-Channel (CTC) clustering, a shared DASD configuration is enhanced with two Channel-to-Channel connections linking each pair of systems in the cluster.

This configuration is called a CTC Ring because of the connection topology that results from the paired connections between participating systems.

Z/OS CLUSTERING: PARALLEL SYSPLEX

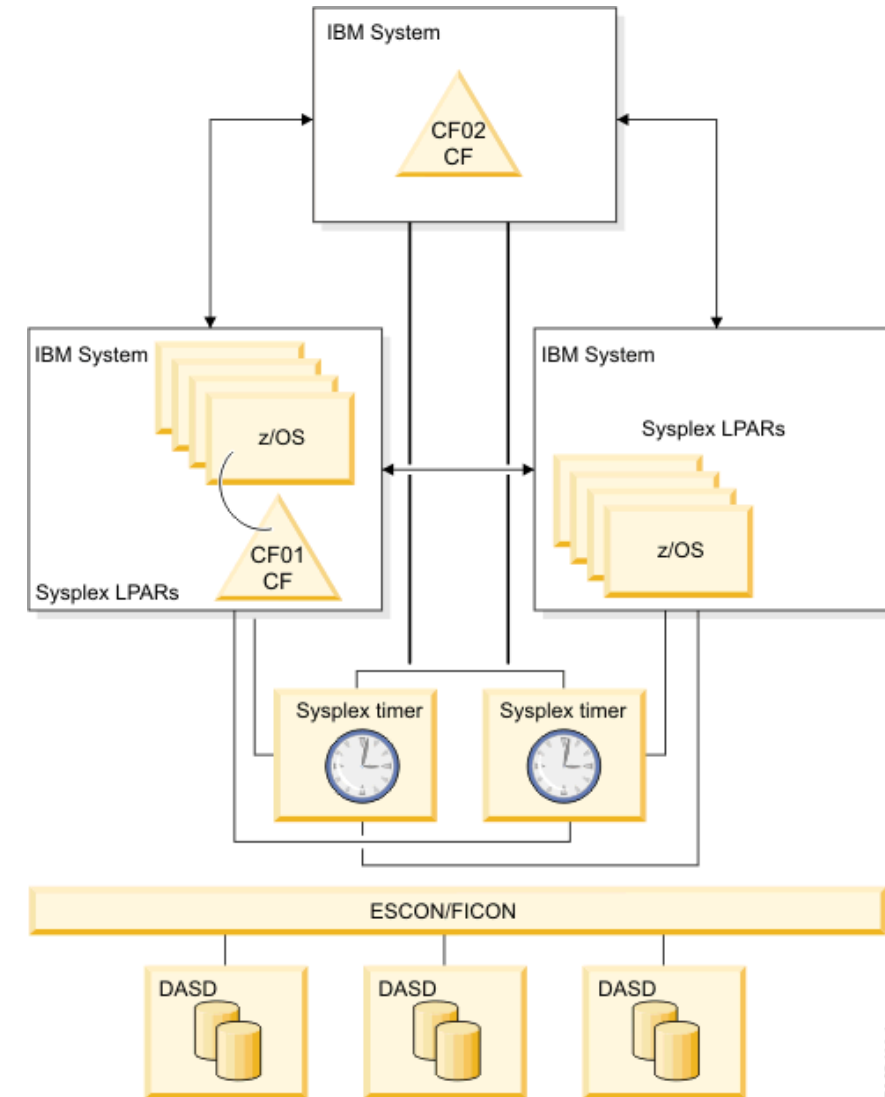
In a Parallel Sysplex, up to 32 z/OS instances can be clustered and operated almost seamlessly as a single system.

What makes it possible is the Coupling Facility (CF). The main purpose of a CF is to enable software running on different systems to share data reliably and safely.

The main services it provides are locking, caching, and list services, through Coupling Facility Control Code (CFCC).

It also manages a single authoritative system clock for all the clustered systems.

A CF runs in a dedicated LPAR. It's an operating system and application in one. The LPAR may be hosted on standalone hardware or on one of the Z systems it manages. It requires processor and microcode support, so it doesn't run on "just any" mainframe hardware.



OPERATING SYSTEM LIMITATIONS

Parallel Sysplex technology is designed to support z/OS LPARs. If you need to include other operating systems in a Parallel Sysplex setup, bear in mind the level of support will be limited, and each non-z/OS operating system has its own peculiar requirements for custom configuration to enable it to participate in a Parallel Sysplex. The following operating systems can be supported to one degree or another, given a list of prerequisites unique to each of them:

- z/VM
- z/VSE
- z/TPF
- Linux

WHAT YOU KNOW AND WHAT YOU CAN DO

- You understand technical information about how z/OS operates and different ways it can be configured.
- You understand the three addressing modes supported by z/OS, for backward compatibility and modernization.
- You know what an address space is and how the virtual memory areas in z/OS address spaces are organized.
- You know about three ways clustering is supported for z/OS, and some of the benefits and limitations of each.

WHAT'S NEXT

- z/OS System Initialization (IPL)